

Reproduction guide and experimental details

Seed-sensitivity and provenance experiments on the LunarLander_RSSM
checkpoint-selection pipeline (arXiv:2607.01736)

Jonathan Shock
University of Cape Town

10 July 2026

1 Summary of runs

Oracle = per-checkpoint CEM-MPC mean return over 20 episodes, MA-7 **centred** smoothing, 100 checkpoints at epochs 5–500. ρ_s is the Spearman correlation of `jac_rof_combined` (averaged over metric seeds 12345/1/2) with the smoothed oracle.

Run	Training data	Actions	Train seed	Peak (epoch)	Collapse	$\rho_s(\text{ROF})$
Your release	human, 750 ep	discrete	12345	+153.0 (310)	yes	−0.707
Reseed	human, 750 ep	discrete	777	+199.2 (495)	no	+0.763
Scripted discrete	scripted, 750 ep	discrete	12345	−43.2 (110)	yes	+0.186
Scripted continuous	scripted, 750 ep	continuous	12345	−391.7 (60)	flat-fail	−0.213*
Mixture f=0.25	188 human + 562 scripted	discrete	12345	+141.5 (155)	—	+0.43
Mixture f=0.75	562 human + 188 scripted	discrete	12345	+144.2 (105)	—	+0.63
Human subset 25%	188 human ep only	discrete	12345	+162.5 (280)	—	−0.11

* Not meaningful: the continuous planner failed at every checkpoint (Section 6), so there was no signal for any metric to predict. The your-release row is my independent re-analysis of your released logs (you report −0.710, my parser gets −0.707, and every published correlation reproduces to within ± 0.004). In the Collapse column, “—” means the flag was not classified in my ledger summary for that cell (it is recomputable from the archived MPC logs).

Validation loss across the same runs: monotone on your human data; turns on the scripted-discrete

run (min at epoch 115, within about 6 points of the oracle checkpoint) and in both mixtures; on the reseed run it correlates at -0.533 in a regime where selection is trivial anyway.

2 Reproducing the seed-777 run

The cell isolates training stochasticity: identical data, identical config, identical evaluation, with only the training seed changed. One structural caveat, from your own released `train_worldmodel_logs.txt`: your run was not one continuous process. It ran fresh to epoch 300 and was then resumed to 500 in a second invocation, and since the pipeline saves only model state, that resume restarted the optimizer moments and re-seeded the RNG stream at epoch 300. My 777 run is a single continuous process, so the comparison is strictly seed-plus-run-structure. This means a couple of things: A fresh, continuous seed-12345 rerun will not bit-reproduce your released checkpoints after epoch 300, so divergence there is expected rather than a failed reproduction. And your collapse onset (peak at 310) sits just after the restart point, which suggests a control worth running: seed 12345 as one continuous process, to separate the restart from the seed.

1. Training. One practical trap on the unmodified code: `train_models.py` auto-resumes from the newest `checkpoints/world_model_*.pt` (setting the start epoch from the filename), and failing that it silently warm-starts from a repo-root `world_model.pt` at epoch 0, which is the worse failure. Move aside both `world_model.pt` and the `checkpoints/` directory first (the shipped `wm_checkpoints/` folder is not read), then:

```
python train_models.py --phase world_model --config config.yaml --seed 777
```

500 epochs, checkpoint every 5, exactly your config (~ 42 s/epoch on an L40S in my run, from the checkpoint timestamps). On my branch the equivalent is `--fresh --checkpoint_dir ckpt_f1rs --seed 777`.

2. MPC sweep. Your own `eval_mpc.py` sweep driver, unchanged (it runs `wm_mpc_policy.py` per checkpoint): 20 episodes per checkpoint, base evaluation seed 12345, all 100 checkpoints, which are exactly its defaults. About 12 h wall-clock on an L40S in my run.

3. Metric sweep. Three full sweeps with seeds 12345, 1, 2, each run start-to-finish in a **single process** (Section 3 explains why this matters), dynamics Jacobians off (they do not enter CROF), then average per checkpoint.

4. Analysis. Centred (not trailing) 7-point moving average over per-checkpoint MPC means; Spearman of seed-averaged `jac_rof_combined` against the smoothed curve. My analysis script (`analyze_llc.py` on the branch) prints the curve characterisation, the full correlation table, and the composite checkpoint picks. You're welcome to it.

2.1 Results

Curve: start +54.7, rising near-monotonically (Spearman vs epoch +0.86) to a peak of +199.2 at epoch 495, the second-to-last checkpoint on the 5–500 grid. Final-10 mean +191.5, median per-checkpoint SE 24.3, no collapse. Full correlation table against the smoothed oracle:

Metric	Pearson	Spearman	R_{quad}^2
emp_C	+0.805	+0.848	0.661
ol_obs_avg	-0.736	-0.775	0.572

Metric	Pearson	Spearman	R_{quad}^2
jac_rof_combined	+0.820	+0.763	0.674
post_obs_rmse	−0.738	−0.745	0.565
jac_rof_bad	+0.792	+0.686	0.627
emp_O	−0.594	−0.670	0.367
jac_ctrl_rank	+0.686	+0.556	0.471
post_rew_rmse	−0.595	−0.541	0.365
val_loss	−0.582	−0.533	0.353
emp_L	+0.457	+0.443	0.234
jac_rof	+0.285	+0.399	0.082
val_kl	+0.496	+0.396	0.247
ol_cumrew_err	−0.339	−0.377	0.115
CROF-B	+0.252	+0.270	0.230
CROF-A	−0.226	−0.158	0.113
jac_obs_rank	−0.186	+0.096	0.245
ol_rew_end	+0.155	+0.061	0.049

Checkpoint picks by the raw composites are poor here, as expected in a monotone regime: CROF-A selects epoch 250 (34% below oracle); CROF-B and jac_rof_combined select epoch 20 (69% below). The oracle answer is effectively the last checkpoint.

3 The RNG stream-position effect

At epoch 5, my L40S run of your unmodified pipeline reproduces your logged metric values exactly. The pipeline is bit-deterministic given the same seed, device class, library versions, and RNG stream position. At epochs 150/280/310/450, my CUDA and CPU values agree with each other to ≤ 0.002 but sit 5–10% above your log (e.g. epoch 280: 0.206 against your 0.186). The reason is stream position, not numerics: your sweep computed all 100 checkpoints in one process, so the RNG stream had advanced by mid-sweep, whereas fresh single-checkpoint runs re-seed. Epoch 5 is the only stream-aligned comparison, and there everything matches exactly.

This means that your logged per-checkpoint values are one consistent stream realisation, reproducible by re-running the full sweep in order. And per-checkpoint Jacobian metrics carry a draw-to-draw sd of about 0.003 ($\approx 4.6\%$ of the metric’s 100-checkpoint range in your run), which is why I average three independent single-process sweeps. The same protocol is what I’d suggest for the 777 reproduction.

4 Scripted-data experiments

Collector. A scripted pilot around Gymnasium’s built-in heuristic with per-episode corruption modes: clean; noisy (action noise $\sigma \sim U[0.4, 0.9]$); blackout (engines cut from $t_0 \sim U(20, 200)$); disoriented (x and v_x sign-flipped as the pilot sees them, from $t_0 \sim U(0, 80)$). Mode probabilities were calibrated against your human dataset’s good/bad/middle return mix only. Discrete version: probabilities 0.26/0.08/0.12/0.54, train 750 episodes / 151,907 steps (mix 43.1/35.2/21.7 against your 41.7/41.5/16.8), val 122 / 24,217, seeds 22345/65432. The collector script is on the branch.

Result. Your untouched discrete pipeline on this data: peak smoothed MPC -43.2 at epoch 110 (raw best $+4.3$), final-10 mean -156.3 , genuine rise-peak-collapse (span, the smoothed curve’s peak minus its minimum: 119.5, against 111 on your run). So at fixed architecture and matched return mix, data provenance moves planning competence by ~ 200 points: human $+153$, scripted discrete -43 , scripted continuous -392 .

Correlations on this run, relevant to your U-shape question. This run collapses, but ROF is not U-shaped in the useful sense: raw `jac_rof_combined` has its minimum at epoch 5, the two pool components point in opposite directions (`jac_rof` $+0.603$, `jac_rof_bad` -0.385), and the combined metric lands at $\rho_s = +0.186$ with its checkpoint pick about 110 points below the oracle checkpoint. Meanwhile `val_loss` outperforms the structural composite here: $\rho_s = -0.434$, turning at epoch 115, within about 6 points of the oracle. So on this evidence the U-shape and the collapse can dissociate, and a U-shaped-ROF collapse detector would miss this collapse while validation loss catches it. (One more lead from your own run, for what it’s worth: your ROF minimum sits at epoch 280 and your MPC peak at 310, so in the one collapsing run where ROF does work, its turn precedes the performance turn by about 30 epochs.)

5 Mixtures and subsets

Episode-level mixtures of your human episodes with the scripted-discrete episodes at human fraction f (composition seed 12345): 750 train episodes each, 122 val episodes mixed at the same f ; plus a human-only subset (188 episodes, full human val set) to separate data diversity from data quantity. A 375-episode subset and an $f=0.5$ mixture were also constructed, but a pre-set phase gate stopped the family before they were trained, so no results exist for them. Results (peak smoothed MPC / val-loss turn / early-stopping gap to oracle / $\rho_s(\text{ROF})$):

Condition	Peak (epoch)	Val-loss turn	ES gap	$\rho_s(\text{ROF})$
$f=0.25$	$+141.5$ (155)	yes	37.9	$+0.43$
$f=0.75$	$+144.2$ (105)	yes	8.8	$+0.63$
human 25%	$+162.5$ (280)	no	47.6	-0.11

Two readings. A 25% human admixture, or 188 human episodes alone, restores planning almost fully (against -43 for pure scripted), so competence is cheap to recover with a small amount of diverse data. And $\rho_s(\text{ROF})$ tracks the shape of each training curve rather than any property of the data, consistent with the seed result.

6 The continuous-action leg, briefly

For completeness, since your limitations section asks about other environments: I ported the pipeline to LunarLanderContinuous (action codec + Gaussian CEM at your planning budget: horizon 25, population 384, elites 48, 4 iterations, $\alpha = 0.7$, $\gamma = 0.97$). Scripted continuous dataset: mode probabilities 0.13/0.15/0.22/0.50, train 750 / 187,627 steps, val 122 / 29,650, seeds 12345/54321; reward-Markovianity gate confirmed comparable to your task ($R^2(o, a) = 0.215$; your protocol reimplemented, and your discrete dataset gives 0.235 under the same reimplementation).

The MPC never worked at any checkpoint (best smoothed -391.7). Diagnosis on the epoch-60 checkpoint: the model is excellent on-manifold (imagined vs real 25-step discounted return under

ground-truth actions: correlation +0.87, mean gap +1.2), but CEM shows an optimism gap of $\approx +50$ (imagines +10 to +15, realises -5 to -58). Random shooting at matched budget finds the same exploited optimum (identical returns and gap), and the environment is solvable through the same loop (heuristic pilot +288 to +299). Done-head gating did nothing, a prior-std uncertainty penalty made it worse, and CEM warm-starting made it much worse (-534 vs -275 at matched seeds). Classic continuous-action model exploitation on narrow data, in other words, and a caution for anyone porting the benchmark to continuous actions with scripted demonstrations.

A Commands

On your unmodified code, for the 777 reproduction (after moving aside the shipped model and checkpoints):

```
python train_models.py --phase world_model --config config.yaml --seed 777
python eval_mpc.py      # sweep driver; defaults are already
                        # 20 eps/ckpt, seed 12345, checkpoints_dir checkpoints
python eval_metrics.py --config config.yaml --seed 12345 \
    --compute_dyn_jac false --output metrics_777_seed12345.txt
    # --compute_dyn_jac false restates the upstream default, for explicitness
    # repeat with --seed 1 and --seed 2, each a full single-process sweep
```

On my branch, the exact forms used for the runs in this document:

```
python train_models.py --phase world_model --config config.yaml --fresh \
    --train_dataset lunarlander_train_dataset.npz \
    --val_dataset lunarlander_val_dataset.npz \
    --checkpoint_dir ckpt_firs --seed 777

python collect_dataset_scripted.py --continuous false --episodes 750 \
    --seed 22345 --p_clean 0.26 --p_noisy 0.08 --p_blackout 0.12 \
    --p_disoriented 0.54 --dataset lunarlander_scripted_train_dataset.npz

python build_mixtures.py      # mixtures/subsets, composition seed 12345

python analyze_llc.py --mpc_log mpc_dq_firs.txt \
    --metrics_logs "metrics_dq_firs_seed*.txt"
```

B Seed registry

Quantity	Seed(s)
Your released training run	pipeline default 12345 (two segments: fresh to epoch 300, resume)
Reseed training	777 (single continuous run)
All other retrained cells	12345
Scripted continuous dataset (train / val)	12345 / 54321
Scripted discrete dataset (train / val)	22345 / 65432
Mixture and subset composition	12345
MPC evaluation (base; episode i uses base+ i)	12345

Quantity	Seed(s)
Metric sweeps (averaged)	12345, 1, 2